glim_localization 使用说明

本文档基于当前工作区中的实际源码、配置和 CMakeLists 编写，适用于开发者和测试人员快速理解、编译、运行和验证 `glim_localization`。

当前工作区相关项目：

- `glim`：GLIM 核心算法库，提供点云预处理、IMU 里程计、因子图优化、submap、地图 dump/load、动态模块加载等基础能力。
- `glim_ros2`：ROS2 封装层，提供 `glim_rosnode` 在线节点和 `glim_rosbag` 离线 rosbag 回放入口。
- `glim_ext`：GLIM 扩展模块集合。当前 `glim_localization` 的重定位实现参考了 `glim_ext` 中 ScanContext loop detector 的思路，但当前代码没有直接链接 `glim_ext`。
- `glim_localization`：基于 GLIM 的 localization-only 子项目，提供新的 odometry 动态模块、地图加载、局部目标地图、scan-to-map registration、轨迹输出、ROS2 localization 输出扩展、重定位和调试工具。

1. 项目简介

`glim_localization` 是一个基于 GLIM 的 localization-only 运行模块。它的目标不是在线构建新地图，而是在已有 GLIM dump 地图上进行实时或离线定位。

它解决的问题：

- 从已有 GLIM 地图 dump 加载 submap。
- 对输入的 LiDAR/IMU 数据执行 GLIM 里程计前端流程。
- 根据预测位姿从固定地图中查询 nearby submaps。
- 构建当前帧的 `LocalTargetMap`。
- 执行当前 scan 到固定地图的 CPU GICP 或可选 GPU VGICP 匹配。
- 将匹配结果作为 pose prior / between factor 接入 GLIM smoother。
- 输出 localization 轨迹、ROS2 pose/odom/status/debug target map 和 TF。
- 在 tracking lost 或服务请求时尝试 ScanContext 风格候选检索 + 几何验证重定位。

与 GLIM full SLAM 的区别：

- full SLAM 以在线建图为主，会持续创建 submap，并运行 local/global mapping。
- `glim_localization` 使用已有地图作为固定参考，不生成新的 submap。
- `glim_localization/config/localization.json` 中明确关闭：
    - `glim_ros.enable_local_mapping = false`
    - `glim_ros.enable_global_mapping = false`
- localization 只加载 fixed map，并将 scan-to-map 匹配结果反馈给 odometry smoother。

当前版本支持的运行模式：

- 离线 rosbag localization：支持。入口是 `glim_ros2` 的 `glim_rosbag`，可使用 `glim_localization/tools/run_offline_localization.sh` 辅助生成临时配置。
- 在线 ROS2 localization：支持。入口是 `glim_ros2` 的 `glim_rosnode`，通过 `libodometry_estimation_localization_cpu.so` 和 `liblocalization_publisher.so` 扩展实现。
- 配置初始位姿：支持。`localization.initial_pose.source = "config"`。
- `/initialpose` 在线初始位姿：支持。`localization.initial_pose.source = "topic"` 时，模块会等待 `/initialpose`。
- 重定位：当前版本已实现基础能力。tracking lost 或调用 `/localization/relocalize` 后，会使用 ScanContext 风格候选检索，并复用 registration 做几何验证。当前实现属于 MVP/P3 版本，不包含完整多假设管理或复杂 smoother 重置策略。
- 独立 `glim_localization_ros` 包：当前版本未实现。ROS2 在线输出通过 `glim_ros2` extension 完成。

2. 项目结构说明

当前 `glim_localization` 目录结构：

```
glim_localization/
  CMakeLists.txt
  package.xml
  config/
    config.json
    localization.json
  include/glim_localization/
    core/
    initialization/
    map/
    map_loader/
    output/
    registration/
    relocalization/
  src/glim_localization/
    core/
    initialization/
    map/
    map_loader/
    output/
    registration/
    relocalization/
  modules/
    odometry/localization_cpu/
    extension/localization_publisher/
  tests/
  tools/
```

关键目录：

- `config/`
  - `config.json`：GLIM 全局配置索引文件，将 `config_ros`、`config_odometry`、`config_localization` 等都映射到 `localization.json`。
  - `localization.json`：localization-only 主配置。
- `include/glim_localization/core/`
  - `localization_options.hpp`：配置结构体。
  - `localization_status.hpp`：状态机枚举。

- `include/glim_localization/initialization/`
    - `config_initial_pose_provider.hpp`：从配置读取初始位姿。
    - `runtime_initial_pose.hpp`：运行时 `/initialpose` 缓存。
- `include/glim_localization/map/`
    - `localization_map.hpp`：全局只读地图对象，持有 submap 列表，支持 nearby query。
    - `local_target_map.hpp`：当前帧局部目标地图，保存 active submap ids、target center 和缓存状态。
    - `submap_index.hpp`：大地图 submap 空间索引。
- `include/glim_localization/map_loader/`
    - `map_format_checker.hpp`：检查 GLIM dump 格式。
    - `glim_map_loader.hpp`：加载 GLIM dump submaps 为 `LocalizationMap`。
- `include/glim_localization/registration/`
    - `map_registration_base.hpp`：scan-to-map registration 抽象接口。
    - `cpu_gicp_map_registration.hpp`：CPU GICP direct scan-to-map。
    - `gpu_vgicp_map_registration.hpp`：可选 GPU VGICP direct scan-to-map。
- `include/glim_localization/relocalization/`
    - `scan_context_relocalizer.hpp`：ScanContext 风格候选检索。
    - `geometric_verifier.hpp`：候选位姿几何验证，复用 registration。
    - `runtime_relocalization_request.hpp`：运行时重定位请求标志。
- `modules/odometry/localization_cpu/`
    - `odometry_estimation_localization_cpu.cpp`：核心 localization odometry module。
    - `odometry_estimation_localization_cpu_create.cpp`：GLIM 动态模块入口。
- `modules/extension/localization_publisher/`
    - `localization_publisher.cpp`：ROS2 输出扩展，发布 status、pose、odom、TF、debug target map，并订阅 `/initialpose`。
- `tools/`
    - `glim_localization_map_info.cpp`：地图格式检查和地图信息打印工具。
    - `benchmark_localization.cpp`：nearby query / index 简单 benchmark。

    - `run_offline_localization.sh`：离线 rosbag localization 辅助脚本。
- `tests/`
    - 单元测试与小型功能验证入口。

与其他项目的关系：

- 依赖 `glim`
    - `glim_localization` 链接 `glim::glim`。
    - localization odometry 类继承并复用 `glim::OdometryEstimationIMU`。
    - 地图加载复用 `glim::SubMap::load()`。
    - ROS2 输出扩展继承 `glim::ExtensionModuleROS2`。
    - 动态 odometry 模块入口符合 GLIM 插件机制：`create_odometry_estimation_module()`。
- 依赖 `glim_ros2`
    - 在线运行使用 `ros2 run glim_ros glim_rosnode`。
    - 离线 rosbag 使用 `ros2 run glim_ros glim_rosbag <bag>`。
    - `glim_ros2` 通过配置中的 `odometry_estimation.so_name` 加载 `libodometry_estimation_localization_cpu.so`。
    - `glim_ros2` 通过 `glim_ros.extension_modules` 加载 `liblocalization_publisher.so`。
- 与 `glim_ext`
    - 当前 `glim_localization` 的 relocalization 参考 ScanContext 思路，但当前 CMake 没有直接 `find_package(glim_ext)` 或链接 `glim_ext`。
    - 当前版本不要求 `glim_ext` 作为编译依赖。

3. 环境依赖

编译依赖来自 `glim_localization/CMakeLists.txt` 和 `package.xml`：

- C++17
- CMake >= 3.16
- `glim`
- `gtsam_points`
- ROS2 环境下：
    - `ament_cmake`
    - `rclcpp`

    - `std_msgs`
    - `std_srvs`
    - `nav_msgs`
    - `sensor_msgs`
    - `geometry_msgs`
    - `tf2_ros`

运行依赖：

- 已构建并 source 的 GLIM/ROS2 workspace。
- `glim_ros2` 包，用于运行 `glim_rosnode` 和 `glim_rosbag`。
- `libodometry_estimation_localization_cpu.so` 必须在运行时可被 GLIM 动态加载。
- 在线输出需要 `liblocalization_publisher.so` 可被 GLIM extension loader 加载。
- 地图必须是 GLIM dump 目录，详见“地图准备”。

可选 GPU 依赖：

- `CMakeLists.txt` 提供 `BUILD_WITH_CUDA` 选项，默认 `OFF`。
- 只有当 `BUILD_WITH_CUDA=ON` 且 `GTSAM_POINTS_USE_CUDA` 可用时，才会定义 `GLIM_LOCALIZATION_USE_CUDA` 并编译 `gpu_vgicp_map_registration.cpp`。
- 如果配置 `localization.matching.method = "gpu_vgicp"`，但构建时没有 CUDA 支持，当前 odometry module 会回退到 CPU GICP，并打印 warning。

注意事项：

- 当前 `package.xml` 中 `maintainer` 仍是 `TODO`，这是项目元信息待完善项，不影响本地开发编译。
- 当前没有单独的 `glim_localization_ros` 包。
- 当前 ROS2 可执行入口来自 `glim_ros2` 包名 `glim_ros`，不是 `glim_ros2`。

4. 编译方法

4.1 前置准备

建议工作区结构类似：

```
<ws>/
  src/ 或当前根目录/
    glim/
    glim_ros2/
    glim_ext/
    glim_localization/
```

需要先确保 `glim`、`glim_ros2` 以及 `gtsam_points` 等依赖能够正常编译。具体第三方依赖安装方式以 `glim` 原项目说明为准。

4.2 编译 ROS2 版本

在包含这些包的 colcon workspace 根目录执行：

```
colcon build --symlink-install --packages-up-to glim_ros glim_localization
source install/setup.bash
```

如果只重新编译 `glim_localization`：

```
colcon build --symlink-install --packages-select glim_localization
source install/setup.bash
```

4.3 编译可选 GPU registration

```
colcon build --symlink-install --packages-select glim_localization --cmake-args -DBUILD_WI
source install/setup.bash
```

如果 `gtsam_points` 没有启用 CUDA，CMake 会打印 warning，GPU VGICP 不会编译，运行时会回退 CPU。

4.4 确认产物

编译安装后，重点检查以下文件：

```
ls install/glim_localization/lib/libglim_localization.so
ls install/glim_localization/lib/libodometry_estimation_localization_cpu.so
ls install/glim_localization/lib/liblocalization_publisher.so
ls install/glim_localization/lib/glim_localization/glim_localization_map_info
ls install/glim_localization/lib/glim_localization/benchmark_localization
```

其中：

- `libglim_localization.so`：localization 公共库。
- `libodometry_estimation_localization_cpu.so`：GLIM odometry 动态模块。
- `liblocalization_publisher.so`：GLIM ROS2 extension 动态模块。
- `glim_localization_map_info`：地图检查工具。
- `benchmark_localization`：submap query benchmark 工具。

4.5 常见编译失败原因

- `find_package(glim REQUIRED)` 失败：`glim` 未安装或未 source 对应 workspace。
- `find_package(gtsam_points REQUIRED)` 失败：`gtsam_points` 未安装或未被 CMake 找到。
- ROS2 消息包找不到：没有 source ROS2 环境，或缺少 `nav_msgs`、`sensor_msgs`、`geometry_msgs`、`tf2_ros`。
- ROS2/ament 配置阶段报 `ModuleNotFoundError: No module named 'catkin_pkg'`：常见于 CMake 误用了 Conda Python。优先退出 Conda 环境，或显式添加 `-DPython3_EXECUTABLE=/usr/bin/python3`。
- 链接阶段出现 `spdlog` / `fmt` ABI 或 `fmt::v8` / `fmt::v9` 混用错误：通常是系统库和 Conda 库混用。建议避免把 `miniconda3/bin` 放在本次构建 PATH 前面，或在干净 shell 中重新配置并编译。
- `liblocalization_publisher.so` 没有生成：当前环境没有设置 `ROS_VERSION=2`，或不是 ROS2 构建环境。
- GPU 文件编译失败：CUDA/gtsam_points CUDA 版本不匹配。可先关闭 `BUILD_WITH_CUDA`。

5. 配置说明

`glim_localization/config/config.json` 是 GLIM 的全局配置入口：

```
{
  "global": {
    "config_path": "",
    "config_ros": "localization.json",
    "config_logging": "localization.json",
    "config_viewer": "localization.json",
    "config_sensors": "localization.json",
    "config_preprocess": "localization.json",
    "config_odometry": "localization.json",
    "config_sub_mapping": "localization.json",
    "config_global_mapping": "localization.json",
    "config_localization": "localization.json"
  }
}
```

运行时需要将 `config_path` 指向包含 `config.json` 和 `localization.json` 的目录。

注意：`glim_ros2` 中 `GlimROS` 的默认 `config_path` 是 `"config"`。如果传入相对路径，它会按 `share/glim/<config_path>` 解析。因此运行 localization 时建议传绝对路径。

5.1 当前 `localization.json` 完整示例

```json
{
  "glim_ros": {
    "imu_topic": "/imu",
    "points_topic": "/points",
    "image_topic": "",
    "keep_raw_points": false,
    "imu_time_offset": 0.0,
    "points_time_offset": 0.0,
    "acc_scale": 0.0,
    "enable_local_mapping": false,
    "enable_global_mapping": false,
    "extension_modules": ["liblocalization_publisher.so"]
  },
  "odometry_estimation": {
    "so_name": "libodometry_estimation_localization_cpu.so",
    "initialization_mode": "LOOSE",
    "smoother_lag": 5.0,
    "fix_imu_bias": false,
    "validate_imu": true,
    "save_imu_rate_trajectory": false,
    "num_threads": 4
  },
  "localization": {
    "map_path": "",
    "map": {
      "load_voxelmaps": true,
      "load_raw_frames": false,
      "strict": true
    },
    "trajectory_path": "/tmp/glim_localization_traj.txt",
    "map_frame": "map",
    "odom_frame": "odom",
    "base_frame": "base_link",
    "sensor_frame": "lidar",
    "initial_pose": {
      "source": "config",
      "xyz": [0.0, 0.0, 0.0],
      "rpy": [0.0, 0.0, 0.0]
    },
```

```
  "target_map": {
    "max_num_submaps": 8,
    "max_distance": 40.0,
    "update_distance": 2.0,
    "update_angle": 0.2,
    "use_submap_index": true,
    "index_resolution": 20.0
  },
  "matching": {
    "method": "cpu_gicp",
    "max_iterations": 20,
    "min_score": 0.35,
    "min_inliers": 30,
    "max_residual": 1000000.0,
    "max_pose_correction_translation": 3.0,
    "max_pose_correction_angle": 0.7,
    "max_consecutive_rejections": 3,
    "max_correspondence_distance": 2.0,
    "pose_prior_precision": 1000.0,
    "num_threads": 4,
    "vgicp_resolution": 0.5,
    "vgicp_voxelmap_levels": 2,
    "vgicp_voxelmap_scaling_factor": 2.0
  },
  "relocalization": {
    "enable": true,
    "max_candidates": 5,
    "num_rings": 20,
    "num_sectors": 60,
    "min_radius": 1.0,
    "max_radius": 80.0,
    "max_descriptor_distance": 0.35
  },
  "ros": {
    "publish_tf": true,
    "publish_debug_target_map": true,
    "initial_pose_topic": "/initialpose",
    "relocalization_service": "/localization/relocalize",
    "status_topic": "/localization/status",
```

```
      "odom_topic": "/localization/odom",
      "pose_topic": "/localization/pose",
      "target_map_topic": "/localization/target_map"
    }
  },
  "sensors": {
    "T_lidar_imu": [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0],
    "imu_bias_noise": 0.001,
    "imu_bias": [0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
    "intensity_field": "intensity",
    "ring_field": ""
  },
  "preprocess": {
  },
  "logging": {
  },
  "viewer": {
  },
  "sub_mapping": {
    "so_name": ""
  },
  "global_mapping": {
    "so_name": ""
  }
}
```

5.2 关键配置项

`glim_ros`：

- `imu_topic`：IMU 输入 topic。
- `points_topic`：点云输入 topic。
- `image_topic`：图像 topic。当前 localization 主流程不依赖图像，默认空字符串。
- `enable_local_mapping`：必须为 `false`，避免运行 GLIM local mapping。
- `enable_global_mapping`：必须为 `false`，避免运行 GLIM global mapping。
- `extension_modules`：当前配置加载 `liblocalization_publisher.so`，用于 ROS2 localization 输出。

`odometry_estimation`：

- `so_name`：必须设置为 `libodometry_estimation_localization_cpu.so`，GLIM 会通过该动态库创建 odometry module。
- `smoother_lag`、`fix_imu_bias`、`validate_imu` 等参数由 GLIM 原始 IMU odometry 使用。

`localization.map_path`：

- GLIM dump 地图目录。
- 不能为空，否则 odometry module 会打印 `localization.map_path is empty; localization map is not loaded` 并进入 `WAIT_MAP`。

`localization.trajectory_path`：

- localization 轨迹输出路径。
- 默认 `/tmp/glim_localization_traj.txt`。

`localization.initial_pose`：

- `source = "config"`：启动时从 `xyz` 和 `rpy` 读取初始 `T_map_imu`。
- `source = "topic"`：启动后等待 `localization.ros.initial_pose_topic`，默认 `/initialpose`。
- `xyz` 单位为米。
- `rpy` 单位为弧度，按 roll、pitch、yaw 设置。

`localization.target_map`：

- `max_num_submaps`：每次最多选取多少个 nearby submaps。
- `max_distance`：submap center 查询最大距离。
- `update_distance`：位移超过该阈值才重建 target map。
- `update_angle`：旋转超过该阈值才重建 target map。
- `use_submap_index`：是否构建 submap 空间索引。
- `index_resolution`：索引栅格分辨率。

`localization.matching`：

- `method`：`cpu_gicp` 或 `gpu_vgicp`。`gpu_vgicp` 需要编译时启用 CUDA，否则

回退 CPU。

- `max_iterations`：registration 优化迭代次数。
- `min_score`：匹配接受的最低 score。
- `min_inliers`：匹配接受的最低 inlier 数。
- `max_residual`：最大 residual。
- `max_pose_correction_translation`：最大平移修正量。
- `max_pose_correction_angle`：最大旋转修正量，单位弧度。
- `max_consecutive_rejections`：连续拒绝次数超过阈值后进入 LOST，并尝试重定位。
- `max_correspondence_distance`：CPU GICP correspondence 距离。
- `pose_prior_precision`：将匹配结果加入 smoother 的 prior / between factor precision。
- `num_threads`：CPU registration 线程数。
- `vgicp_*`：GPU VGICP 使用的参数。

`localization.relocalization`：

- `enable`：是否启用重定位。
- `max_candidates`：候选 submap 数。
- `num_rings`、`num_sectors`、`min_radius`、`max_radius`：ScanContext 风格描述子参数。
- `max_descriptor_distance`：候选描述子最大距离。

`localization.ros`：

- `publish_tf`：是否发布 TF。
- `publish_debug_target_map`：是否发布 debug target map 点云。
- `initial_pose_topic`：初始位姿 topic，默认 `/initialpose`。
- `relocalization_service`：重定位触发服务，默认 `/localization/relocalize`。
- `status_topic`：状态输出 topic。
- `odom_topic`：odom 输出 topic。
- `pose_topic`：pose 输出 topic。
- `target_map_topic`：debug target map 输出 topic。

`sensors`：

- `T_lidar_imu`：LiDAR 到 IMU 的外参，格式为 `[x, y, z, qx, qy, qz, qw]`。该参数由 GLIM 原始传感器配置读取。
- `imu_bias_noise`、`imu_bias`：IMU bias 配置。
- `intensity_field`、`ring_field`：点云字段名。

为什么要关闭 local/global mapping：

- localization-only 使用已有地图，不应该创建新 submap。
- 如果开启 local/global mapping，GLIM 会尝试运行建图模块，可能生成新地图、额外线程和不符合 localization-only 的输出。
- 当前 `glim_ros2` 实际读取的是 `glim_ros.enable_local_mapping` 和 `glim_ros.enable_global_mapping`，因此这两个字段必须保持 `false`。

6. 地图准备

当前版本支持的地图输入：

- GLIM dump 目录。
- 通过 `glim::SubMap::load()` 加载 numbered submap 目录。
- 当前 loader 不把完整 `GlobalMapping` 作为运行期核心对象，也不依赖在线 global mapping。

地图目录要求：

```
<map_path>/
  graph.txt
  graph.bin        # 可选，缺失会 warning
  values.bin       # 可选，缺失会 warning
  000000/
    data.txt
    ...            # SubMap::load() 需要的点云/子地图文件
  000001/
    data.txt
    ...
  ...
```

`map_format_checker.cpp` 当前检查逻辑：

- `map_path` 不能为空。
- `map_path` 必须存在且是目录。
- 必须存在 `graph.txt` 。
- `graph.txt` 必须按顺序包含：
    - `num_submaps: <non-negative int>`
    - `num_all_frames: <non-negative int>`
    - `num_matching_cost_factors: <non-negative int>`
- `graph.bin` 缺失会 warning，不会直接导致 invalid。
- `values.bin` 缺失会 warning，不会直接导致 invalid。
- 根据 `num_submaps` 检查 `000000` 、 `000001` 等 submap 目录。
- 每个 submap 目录必须包含 `data.txt` 。

地图加载逻辑：

- `GlimMapLoader::load()` 先调用 `MapFormatChecker` 。
- 对每个 submap 路径 `<map_path>/%06d` 调用 `glim::SubMap::load()` 。
- 成功后返回 `std::shared_ptr<LocalizationMap>` 。
- `LocalizationMap` 持有只读 submap 列表。

6.1 使用 map info 工具检查地图

安装后工具路径：

```
ros2 run glim_localization glim_localization_map_info <map_path> [index_resolution]
```

示例：

```
ros2 run glim_localization glim_localization_map_info /data/maps/glim_dump 20.0
```

成功时会输出：

- `valid: true`
- `num_submaps`
- `num_all_frames`
- `num_matching_cost_factors`
- `loaded_submaps`

- `merged_submap_points`
- submap origin 范围
- submap index 统计

如果没有安装，也可以从 build 目录运行，具体路径需根据本地 colcon/CMake 生成目录调整，例如：

```
./build/glim_localization/glim_localization_map_info /data/maps/glim_dump 20.0
```

6.2 人工检查建议

如果地图加载失败，先检查：

```
ls /data/maps/glim_dump/graph.txt
head -n 3 /data/maps/glim_dump/graph.txt
ls /data/maps/glim_dump/000000/data.txt
```

`graph.txt` 的前三行必须符合 checker 当前解析格式。

7. 离线运行说明

离线模式使用 `glim_ros2` 的 `glim_rosbag` 回放 rosbag，并通过配置加载 localization odometry module。

需要输入：

- GLIM dump 地图目录。
- ROS2 rosbag 路径，bag 中至少应包含配置中的：
  - `glim_ros.imu_topic`，默认 `/imu`
  - `glim_ros.points_topic`，默认 `/points`
- 初始位姿 `T_map_imu`，可通过脚本参数写入临时配置。
- 已 source 编译后的 workspace。

7.1 推荐方式：使用源码树脚本

当前脚本：

```
glim_localization/tools/run_offline_localization.sh
```

用法来自脚本源码：

```
bash glim_localization/tools/run_offline_localization.sh \
  <rosbag_path> \
  <map_path> \
  <x> <y> <z> <roll> <pitch> <yaw> \
  [trajectory_path]
```

示例：

```
source install/setup.bash

bash glim_localization/tools/run_offline_localization.sh \
  /data/bags/sample_bag \
  /data/maps/glim_dump \
  0.0 0.0 0.0 0.0 0.0 0.0 \
  /tmp/glim_localization_traj.txt
```

脚本做的事情：

- 复制 `glim_localization/config/` 到 `/tmp/glim_localization_config.XXXXXX`。
- 修改临时 `localization.json`：
  - `glim_ros.enable_local_mapping = false`
  - `glim_ros.enable_global_mapping = false`
  - `odometry_estimation.so_name = "libodometry_estimation_localization_cpu`
  - `localization.map_path = <map_path>`
  - `localization.trajectory_path = <trajectory_path>`
  - `localization.initial_pose.source = "config"`
  - `localization.initial_pose.xyz/rpy = 脚本参数`
- 调用：

```
ros2 run glim_ros glim_rosbag "<rosbag_path>" --ros-args -p "config_path:=<临时配置目录>"
```

当前脚本已经兼容源码树和安装后的 ROS2 布局。下面两种方式都可用：

- 源码树：`bash glim_localization/tools/run_offline_localization.sh ...`
- 安装后：`install/glim_localization/lib/glim_localization/run_offline_localization.sh ...`

7.2 直接运行 glim_rosbag

也可以手动准备一个配置目录，然后运行：

```
source install/setup.bash

ros2 run glim_ros glim_rosbag /data/bags/sample_bag \
  --ros-args -p config_path:=/absolute/path/to/glim_localization/config
```

如果使用源码目录配置，需要先编辑：

- `glim_localization/config/localization.json` 中的 `localization.map_path`
- `localization.trajectory_path`
- `localization.initial_pose`

`glim_rosbag` 源码中还支持这些 ROS 参数：

- `delay`
- `start_offset`
- `playback_duration`
- `playback_until`
- `end_time`

例如：

```
ros2 run glim_ros glim_rosbag /data/bags/sample_bag \
  --ros-args \
  -p config_path:=/absolute/path/to/glim_localization/config \
  -p start_offset:=5.0 \
  -p playback_duration:=30.0
```

7.3 成功运行时的关键日志

日志中应能看到类似信息：

- `localization trajectory output: /tmp/glim_localization_traj.txt`
- `valid GLIM map dump: <N> submaps in <map_path>`
- `loaded localization map: <N> submaps`
- `localization map loaded: <N> submaps`
- `localization submap index built: ...`
- `using config initial pose for localization (world == map)`
- `localization publisher extension initialized`
- `rebuilt localization target map: ... active_submaps=[...]`
- `scan-to-map registration accepted ...`

如果连续匹配失败，可能看到：

- `scan-to-map registration rejected ...`
- `relocalization query frame=...`
- `relocalization succeeded ...`
- 或 `relocalization failed: ...`

7.4 离线输出

默认轨迹文件：

```
/tmp/glim_localization_traj.txt
```

也可通过 `localization.trajectory_path` 或脚本最后一个参数修改。

判断离线运行成功：

- odometry module 成功加载。
- map 成功加载。
- 有 target map rebuild/reuse 日志。
- 有 registration accepted 日志。
- 轨迹文件存在且有多行结果。
- 没有 local/global mapping 生成新 submap 的日志。

8. 在线 ROS2 运行说明

在线模式使用 `glim_ros2` 的 `glim_rosnode`：

```
source install/setup.bash

ros2 run glim_ros glim_rosnode \
  --ros-args -p config_path:=/absolute/path/to/glim_localization/config
```

运行前请编辑 `localization.json`：

- 设置 `localization.map_path`。
- 根据传感器设置 `glim_ros.imu_topic` 和 `glim_ros.points_topic`。
- 设置正确的 `sensors.T_lidar_imu`。
- 设置初始位姿方式：
    - 使用配置：`localization.initial_pose.source = "config"`
    - 使用 RViz `/initialpose`：`localization.initial_pose.source = "topic"`

8.1 订阅输入

由 `glim_ros2` 主节点订阅：

- `/imu`，来自 `glim_ros.imu_topic`
- `/points`，来自 `glim_ros.points_topic`
- `image_topic` 默认空字符串，当前 localization 不依赖图像

由 `liblocalization_publisher.so` 扩展订阅：

- `/initialpose`，消息类型 `geometry_msgs/msg/PoseWithCovarianceStamped`

8.2 发布输出

由 `liblocalization_publisher.so` 发布：

- `/localization/status`
    - 类型：`std_msgs/msg/String`
    - 内容形如：`TRACKING score=0.823000`

- `/localization/odom`
    - 类型：`nav_msgs/msg/Odometry`
    - `header.frame_id = odom`
    - `child_frame_id = base_link`
- `/localization/pose`
    - 类型：`geometry_msgs/msg/PoseStamped`
    - `header.frame_id = map`
- `/localization/target_map`
    - 类型：`sensor_msgs/msg/PointCloud2`
    - frame 为 `map`
    - 仅当 `publish_debug_target_map = true` 且 topic 有订阅者时构建并发布

服务：

- `/localization/relocalize`
    - 类型：`std_srvs/srv/Trigger`
    - 调用后设置运行时 relocalization request，下一帧处理时尝试重定位。

TF：

- `map -> odom`
- `odom -> base_link`

当前实现中 `T_map_odom` 初始化为 identity，并在 P2/P3 代码中作为预留变量使用。当前没有实现复杂的 `map->odom` 连续重置策略。

8.3 `/initialpose` 使用

如果配置：

```
"initial_pose": {
  "source": "topic",
  "xyz": [0.0, 0.0, 0.0],
  "rpy": [0.0, 0.0, 0.0]
}
```

odometry module 在收到 `/initialpose` 前会等待，并打印：

```
waiting for /initialpose before starting localization
```

可以在 RViz 中使用 `2D Pose Estimate` 发布 `/initialpose`，也可以命令行发布，例如：

```
ros2 topic pub --once /initialpose geometry_msgs/msg/PoseWithCovarianceStamped "{
  header: {frame_id: 'map'},
  pose: {
    pose: {
      position: {x: 0.0, y: 0.0, z: 0.0},
      orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}
    }
  }
}"
```

8.4 在线功能边界

当前已实现：

- 在线 topic 输入复用 `glim_rosnode`。
- `/initialpose` 订阅。
- status / pose / odom / TF 发布。
- debug target map 发布。
- `/localization/relocalize` 服务。

当前未实现或待补充：

- 独立 `glim_localization_ros` 包。
- 完整生命周期节点或 launch 文件。
- 完整 `map->odom` reset 管理。
- RViz 配置文件。
- 与外部定位系统的标准 covariance 输出，目前 publisher 没有填充完整 pose covariance。

9. 输出说明

9.1 轨迹文件格式

轨迹由 `TrajectoryWriter` 写出，每行格式：

```
stamp x y z qx qy qz qw status_int matching_score
```

字段说明：

- `stamp`：帧时间戳，秒。
- `x y z`：`T_map_imu` 平移。
- `qx qy qz qw`：`T_map_imu` 旋转四元数。
- `status_int`：`LocalizationStatus` 枚举整数值。
- `matching_score`：最近一次 registration score。

状态枚举顺序来自 `localization_status.hpp`：

```
0 WAIT_MAP
1 WAIT_INITIAL_POSE
2 INITIALIZING
3 TRACKING
4 LOST
5 RELOCALIZING
```

默认路径：

```
/tmp/glim_localization_traj.txt
```

9.2 ROS 输出

`/localization/status`：

- `std_msgs/msg/String`
- 内容是状态名和 score，例如 `TRACKING score=0.900000`。

`/localization/pose`：

- `geometry_msgs/msg/PoseStamped`
- frame 为 `map`
- pose 为 `T_map_imu`

`/localization/odom`：

- `nav_msgs/msg/Odometry`
- frame 为 `odom`
- child frame 为 `base_link`
- 当前 pose 为 `T_odom_imu = T_map_odom.inverse() * T_map_imu`
- twist linear 来自 GLIM estimation frame 的 `v_world_imu`

`/localization/target_map`：

- `sensor_msgs/msg/PointCloud2`
- frame 为 `map`
- 内容为 active submaps 的点云转换到 map frame 后合并。

9.3 坐标系约定

当前实现中：

- `world == map`
- `frame->T_world_imu == T_map_imu`
- `frame->T_world_lidar == T_map_lidar`
- `T_map_lidar = T_map_imu * T_lidar_imu.inverse()`
- `T_map_odom` 当前默认为 identity，预留给后续 reset/relocalization 维护。

配置中的 frame 名：

- `map_frame = "map"`
- `odom_frame = "odom"`
- `base_frame = "base_link"`
- `sensor_frame = "lidar"`

注意：

- 当前 `/localization/odom` 和 `odom -> base_link` 使用的是 IMU 位姿，但 child frame 名配置为 `base_link`。
- 如果系统中 `base_link`、`imu`、`lidar` 不是同一坐标系，需要由上层 TF 或配置外参保证一致。
- 本包不发布 `base_link -> imu` 或 `base_link -> lidar` 静态 TF。

10. 测试与验证

10.1 运行单元测试

编译测试：

```
colcon build --symlink-install --packages-select glim_localization --cmake-args -DBUILD_TE
```

运行测试：

```
colcon test --packages-select glim_localization
colcon test-result --verbose
```

也可以在 build 目录直接执行测试可执行文件，路径需根据本地构建目录调整。

当前测试：

- `test_map_format_checker`
  - 测试合法/非法 GLIM dump 路径和 `graph.txt`/submap 目录检查。
- `test_glim_map_loader`
  - 如果没有提供真实地图 fixture，会输出 `GLIM_LOCALIZATION_TEST_MAP is not set; skipping real GLIM dump loading` 并跳过。
  - 可通过环境变量指定真实 GLIM dump：

```
GLIM_LOCALIZATION_TEST_MAP=/data/maps/glim_dump colcon test --packages-select glim_localiz
```

- `test_localization_map`
  - 使用人工 submap pose 验证 nearby query、最大距离、最大数量和

active submap id。

- `test_submap_index`
  - 验证 submap 空间索引。
- `test_cpu_map_registration`
  - 使用小型 scan + target map 验证 CPU GICP 匹配输出。
- `test_scan_context_relocalizer`
  - 验证 ScanContext 风格 relocalizer 基础查询。

10.2 最小闭环验证

推荐顺序：

1. 验证地图格式：

```
ros2 run glim_localization glim_localization_map_info /data/maps/glim_dump
```

2. 验证 rosbag topic：

```
ros2 bag info /data/bags/sample_bag
```

确认有 `/imu` 和 `/points`，或修改 `localization.json` 中的 topic。

3. 离线运行：

```
bash glim_localization/tools/run_offline_localization.sh \
  /data/bags/sample_bag \
  /data/maps/glim_dump \
  0.0 0.0 0.0 0.0 0.0 0.0 \
  /tmp/glim_localization_traj.txt
```

4. 检查轨迹：

```
wc -l /tmp/glim_localization_traj.txt
head /tmp/glim_localization_traj.txt
tail /tmp/glim_localization_traj.txt
```

5. 检查日志中是否出现：

- map loaded
- target map rebuilt/reused
- scan-to-map registration accepted
- trajectory saved

10.3 benchmark 工具

安装后运行：

```
ros2 run glim_localization benchmark_localization \
  /data/maps/glim_dump \
  1000 \
  40.0 \
  8 \
  20.0
```

参数：

```
benchmark_localization <map_path> [num_queries] [max_distance] [max_num_submaps] [index_re
```

输出包括：

- map load 时间
- linear query 总耗时和平均耗时
- indexed query 总耗时和平均耗时
- index cells 和 max cell size

该工具只 benchmark submap 查询，不运行完整 odometry / registration。

11. 调试建议

11.1 优先检查日志

启动后先看这些关键日志：

- 是否加载了 `libodometry_estimation_localization_cpu.so`
- 是否加载了 `liblocalization_publisher.so`
- `localization.map_path` 是否为空
- `valid GLIM map dump`
- `loaded localization map`
- `using config initial pose` 或 `waiting for /initialpose`
- `rebuilt localization target map`
- `scan-to-map registration accepted/rejected`
- `localization trajectory saved`

11.2 地图加载失败

检查：

- `localization.map_path` 是否是绝对路径。
- 目录是否存在。
- 是否存在 `graph.txt`。
- `graph.txt` 前三行格式是否符合 checker 预期。
- `000000/data.txt` 等 submap 文件是否存在。
- `glim_localization_map_info` 是否能成功加载。

11.3 初始位姿不准

现象：

- registration 大量 rejected。
- status 进入 `LOST`。
- relocalization 失败或频繁触发。

建议：

- 先使用接近地图真实位置的 `initial_pose`。
- 检查 `rpy` 单位是否为弧度。
- 检查 `T_lidar_imu` 是否与建图时一致。
- 放宽 `max_pose_correction_translation`、`max_pose_correction_angle` 仅用于排查，不建议长期过大。

11.4 scan-to-map 不收敛

检查：

- 当前 scan 和 map 是否来自同一传感器配置。
- `max_correspondence_distance` 是否过小。
- `target_map.max_distance` 是否能覆盖足够 submaps。
- `target_map.max_num_submaps` 是否过少。
- `min_score` 和 `min_inliers` 是否过严。
- 地图是否过稀疏或动态物体过多。

11.5 target map 每帧重建导致慢

检查：

- `target_map.update_distance`
- `target_map.update_angle`
- `target_map.use_submap_index`
- `target_map.index_resolution`

当前 `LocalTargetMap` 已支持缓存，只有超过距离或角度阈值才重建。

11.6 topic / TF 异常

检查：

```
ros2 topic list
ros2 topic echo /localization/status
ros2 topic echo /localization/pose
ros2 run tf2_ros tf2_echo map odom
ros2 run tf2_ros tf2_echo odom base_link
```

如果没有 localization 输出：

- 检查 `glim_ros.extension_modules` 是否包含 `liblocalization_publisher.so`。
- 检查 `liblocalization_publisher.so` 是否已编译安装。
- 检查是否已经有输入 `/imu` 和 `/points`。

- 如果 `initial_pose.source = "topic"`，检查是否已经发布 `/initialpose`。

12. 常见问题（FAQ）

Q1：为什么必须提供初始位姿？

当前主流程是 scan-to-map tracking，不是全局定位器。registration 需要一个合理初值来从固定地图中选择 nearby submaps 并收敛。可以通过配置 `localization.initial_pose.source = "config"` 提供，也可以通过 `/initialpose` 提供。

Q2：为什么要关闭 local/global mapping？

localization-only 使用已有地图，不应创建新 submap 或运行全局建图。当前配置中 `glim_ros.enable_local_mapping` 和 `glim_ros.enable_global_mapping` 必须为 `false`。

Q3：地图必须是什么格式？

当前只支持 GLIM dump 目录。必须有 `graph.txt` 和 numbered submap 目录，例如 `000000/data.txt`。`graph.bin` 和 `values.bin` 缺失会 warning，但当前 loader 仍可加载 submaps。

Q4：为什么配置了 `gpu_vgicp` 还是用了 CPU？

只有在 `BUILD_WITH_CUDA=ON` 且 `gtsam_points` 检测到 CUDA 支持时，GPU VGICP 才会编译。否则运行时会打印 warning 并回退 CPU GICP。

Q5：rosbag 能跑但在线不稳定怎么办？

先确认在线 topic 时间戳、IMU/点云同步、外参 `T_lidar_imu`、点云字段名和建图时一致。再检查 `matching` gating 参数和 `target_map` 查询范围。

Q6：如果没有 `/initialpose`，当前怎么启动？

将配置改为：

```
"initial_pose": {
  "source": "config",
  "xyz": [0.0, 0.0, 0.0],
  "rpy": [0.0, 0.0, 0.0]
}
```

并填写实际初始位姿。

Q7：`/localization/target_map` 为什么没有消息？

当前 publisher 只有在 `publish_debug_target_map = true` 且该 topic 有订阅者时才构建并发布 debug cloud。先在 RViz 或命令行订阅该 topic。

Q8：为什么轨迹文件没有生成？

检查 `localization.trajectory_path` 是否为空，路径父目录是否可创建，日志是否有 `failed to open localization trajectory output`。默认路径是 `/tmp/glim_localization_traj.txt`。

Q9：`glim_rosbag` 读取不到配置怎么办？

传入绝对 `config_path`：

```
ros2 run glim_ros glim_rosbag /data/bags/sample_bag \
  --ros-args -p config_path:=/absolute/path/to/glim_localization/config
```

不要依赖相对路径，`glim_ros2` 会把相对路径按 `share/glim` 下的路径解析。

Q10：为什么 status 进入 LOST？

通常是连续 registration rejected 达到 `matching.max_consecutive_rejections`。查看 reject reason，可能是 `low_score`、`few_inliers`、`large_translation_correction`、`large_rotation_correction` 或 optimizer 异常。

Q11：重定位是否已经完整可用？

当前已实现基础重定位：ScanContext 风格候选检索 + registration 几何验证。它可用于 LOST 后恢复 TRACKING，但还不是完整产品级 relocalization：没有复杂多假设跟踪、没有完整 smoother reset 策略。

Q12：能否直接用 `ros2 run glim_localization glim_localization_map_info` ？

可以。当前 ROS2 安装布局已经对齐到 `lib/${PROJECT_NAME}`，在执行过 `source install/setup.bash` 后可直接运行：

```
ros2 run glim_localization glim_localization_map_info <map_path>
```

如需 `ros2 run` 支持，可后续调整 CMake 安装路径到 ROS2 常用的 `lib/${PROJECT_NAME}`。

13. 当前限制与后续计划

13.1 当前已实现能力

- 新增 GLIM odometry 动态模块：
  - `libodometry_estimation_localization_cpu.so`
  - 动态入口：`create_odometry_estimation_module()`
- GLIM dump 地图格式检查：
  - `MapFormatChecker`
- GLIM dump submap 加载：
  - `GlimMapLoader`
  - 复用 `glim::SubMap::load()`
- 只读 localization map：
  - `LocalizationMap`
  - linear nearby query
  - optional `SubmapIndex`
- 当前帧 target map：
  - `LocalTargetMap`

    - active submap ids
    - cache reuse/rebuild
- scan-to-map registration：
    - CPU GICP
    - 可选 GPU VGICP
    - score / residual / inliers / correction gating
- 与 GLIM smoother 集成：
    - accepted matching 写入 pose prior
    - 相邻帧可加入 between factor
- 轨迹输出：
    - `TrajectoryWriter`
- ROS2 输出扩展：
    - status
    - odom
    - pose
    - map->odom TF
    - odom->base_link TF
    - debug target map
    - `/initialpose`
    - `/localization/relocalize`
- 基础重定位：
    - ScanContext 风格候选检索
    - 几何验证
- 工具：
    - `glim_localization_map_info`
    - `benchmark_localization`
    - `run_offline_localization.sh`
- 测试：
    - map checker
    - map loader
    - localization map
    - submap index
    - CPU registration
    - scan context relocalizer

13.2 当前未实现或待补充

- 独立 `glim_localization_ros` 包。
- ROS2 launch 文件。
- RViz 配置。
- 产品级重定位状态管理和 smoother reset 策略。
- 完整 covariance 输出。
- 多地图格式支持，例如 PCD/PLY/自定义语义地图直接导入。
- 完整 GPU 性能验证报告。
- `base_link` 与 IMU pose 语义的进一步解耦。
- `map->odom` 的连续维护策略。

13.3 后续计划建议

基于当前仓库已经存在的代码和 TODO，后续可按以下方向推进：

- P2 稳定性：
    - 继续完善 status 输出和 covariance。
    - 补充 launch/RViz 调试配置。
    - 完善 `T_map_odom` 在重定位后的连续性维护。
- P3 重定位：
    - 增加更鲁棒的候选筛选。
    - 增加 smoother reset 或状态窗口重初始化策略。
    - 增加重定位失败后的降级行为。
- P4 产品化：
    - 增加 benchmark 数据记录和大地图调参指南。
    - 若确有需要，再拆分独立 `glim_localization_ros`。